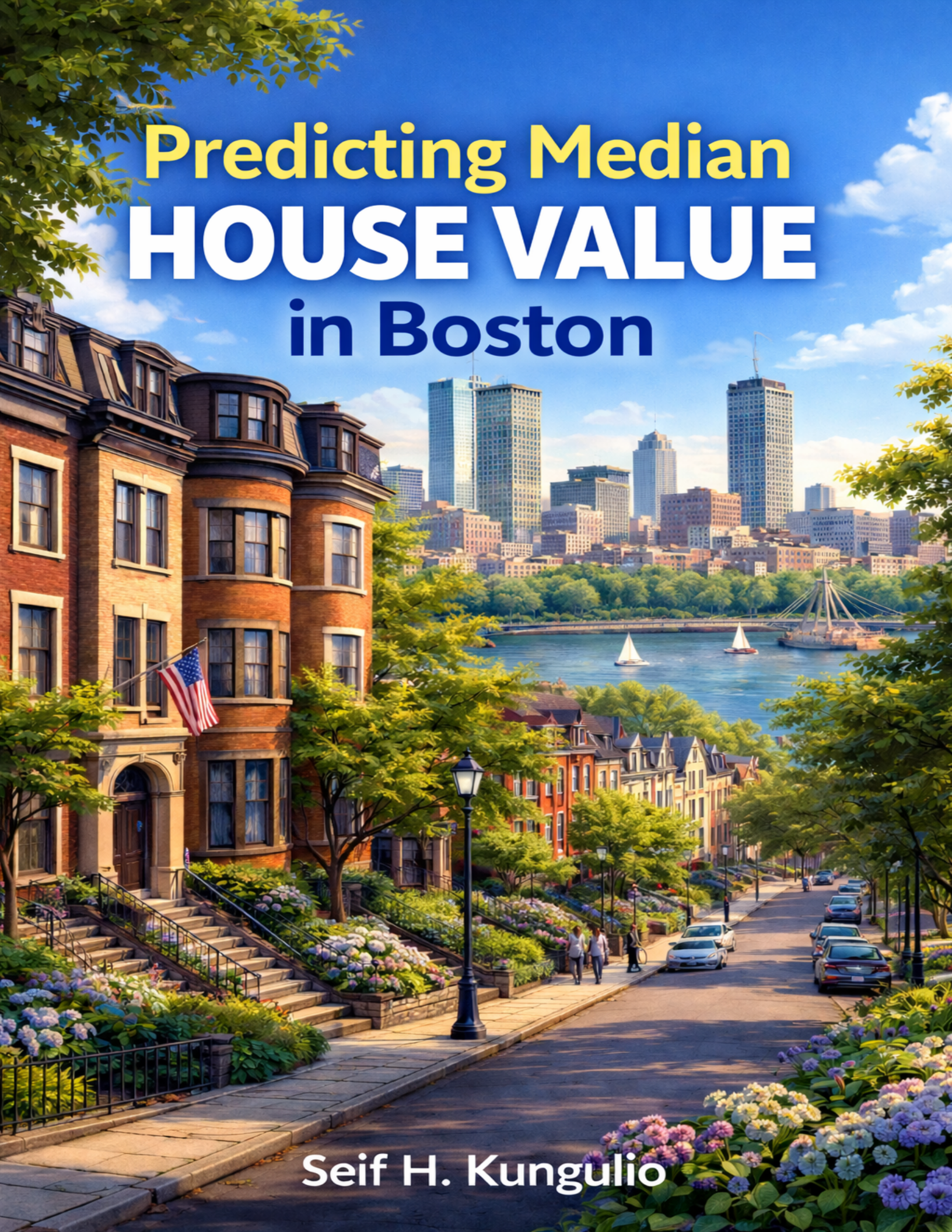
Predicting Median HOUSE VALUE in Boston



Seif H. Kungulio

Predicting Median House Value in Boston

Seif H. Kungulio

February 15, 2026

Contents

Business Understanding

Background & Problem Statement

Boston's housing authority and urban developers face the challenge of accurately estimating property values across different neighborhoods. Property valuation depends on multiple interrelated factors—such as crime rate, accessibility to major roads, environmental quality, and local amenities—which can be difficult to model using traditional statistical techniques alone.

Business Objectives

Build a predictive model to estimate median owner-occupied home value (medv) using neighborhood-level indicators. The goal is to support:

- neighborhood investment planning,
- redevelopment prioritization,
- and value forecasting under changing conditions.

Success Criteria

- **Primary metric:** RMSE (lower is better)
- **Secondary metrics:** MAE and R^2
- **Practical goal:** A model that generalizes well (low validation error) and offers interpretable drivers (feature importance / coefficients).

Data Understanding

Data Collection

The Boston Housing dataset is loaded from the `MASS` package and assigned to the object `boston.df`. A structural inspection is performed using `glimpse()` to verify the number of observations, variable types, and overall dataset organization. This step ensures that the dataset is correctly loaded and ready for analysis.

A descriptive statistical summary of the target variable (medv) is generated to evaluate its range, central tendency, and dispersion. This preliminary assessment provides insight into the scale and distribution of housing values prior to model development.

```
data("Boston", package = "MASS")
boston.df <- Boston

glimpse(boston.df)
```

```
## Rows: 506
## Columns: 14
## $ crim    <dbl> 0.00632, 0.02731, 0.02729, 0.03237, 0.06905, 0.02985, 0.08829,~
## $ zn      <dbl> 18.0, 0.0, 0.0, 0.0, 0.0, 0.0, 12.5, 12.5, 12.5, 12.5, 12.5, 1~
## $ indus   <dbl> 2.31, 7.07, 7.07, 2.18, 2.18, 2.18, 7.87, 7.87, 7.87, 7.87, 7.~
## $ chas    <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ nox     <dbl> 0.538, 0.469, 0.469, 0.458, 0.458, 0.458, 0.524, 0.524, 0.524,~
## $ rm      <dbl> 6.575, 6.421, 7.185, 6.998, 7.147, 6.430, 6.012, 6.172, 5.631,~
## $ age     <dbl> 65.2, 78.9, 61.1, 45.8, 54.2, 58.7, 66.6, 96.1, 100.0, 85.9, 9~
## $ dis     <dbl> 4.0900, 4.9671, 4.9671, 6.0622, 6.0622, 6.0622, 5.5605, 5.9505~
## $ rad     <int> 1, 2, 2, 3, 3, 3, 5, 5, 5, 5, 5, 5, 5, 4, 4, 4, 4, 4, 4, 4, 4,~
## $ tax     <dbl> 296, 242, 242, 222, 222, 222, 311, 311, 311, 311, 311, 311, 31~
## $ ptratio <dbl> 15.3, 17.8, 17.8, 18.7, 18.7, 18.7, 15.2, 15.2, 15.2, 15.2, 15~
## $ black   <dbl> 396.90, 396.90, 392.83, 394.63, 396.90, 394.12, 395.60, 396.90~
## $ lstat   <dbl> 4.98, 9.14, 4.03, 2.94, 5.33, 5.21, 12.43, 19.15, 29.93, 17.10~
## $ medv    <dbl> 24.0, 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15~
```

```
summary(boston.df$medv)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    5.00   17.02   21.20   22.53   25.00   50.00
```

The Boston Housing dataset was loaded from the `MASS` package and inspected to confirm its structural integrity and dimensionality. The dataset contains 506 observations and 14 variables, including 13 predictors and the target variable `medv`, representing median home value in thousands of dollars. Summary statistics reveal that home values range from 5 to 50, with a mean of approximately 22.53 and a median of 21.2. The maximum value of 50 reflects a known ceiling effect in the dataset, where higher values were top-coded. This cap has modeling implications, as it can compress variation among high-priced tracts and potentially lead to systematic underprediction at the upper end of the market.

Data Description

The dataset used in this project is the Boston Housing Dataset, originally published in the Harrison and Rubinfeld (1978) study and included in the MASS package in R. It contains 506 observations and 14 variables

describing socioeconomic, demographic, environmental, and structural characteristics of housing in Boston suburbs. The dataset is commonly used for predictive modeling and regression analysis tasks, particularly for estimating housing values.

Each record represents a census tract in the Boston metropolitan area, with the target variable being the median value of owner-occupied homes (MEDV) expressed in $1000s.

The features cover a wide range of factors that influence property values — including crime rates, industrial activity, environmental quality, accessibility to major roads, and education levels. These variables provide a multi-dimensional view of the housing environment, making the dataset suitable for developing regression and machine learning models to predict housing prices.

Data Dictionary

Variable	Description	Data Type	Constraints/Rule
crim	Per capita crime rate by town	Numeric	>= 0
zn	Proportion of residential land zoned for lots over 25,000 sq.ft.	Numeric	0 <= **zn** <= 100
indus	Proportion of non-retail business acres per town	Numeric	>= 0
chas	Charles River dummy variable (1 if tract bounds river; 0 otherwise)	Integer	0 or 1
nox	Nitric oxides concentration (parts per 10 million)	Numeric	0 < **nox** <= 1
rm	Average number of rooms per dwelling	Numeric	> 0
age	Proportion of owner-occupied units built prior to 1940 (%)	Numeric	0 <= **age** <= 100
dis	Weighted distances to five Boston employment centers	Numeric	> 0
rad	Index of accessibility to radial highways	Integer	1–24
tax	Full-value property-tax rate per $10,000	Numeric	> 0
ptratio	Pupil-teacher ratio by town	Numeric	> 0
black	(1000(Bk - 0.63)^2), where Bk is the proportion of Black residents by town	Numeric	>= 0
lstat	Percentage of lower status of the population	Numeric	0 <= **lstat** <= 100
medv	Median value of owner-occupied homes in $1000s (Target variable)	Numeric	> 0 (typically capped at 50 in dataset)

Data Quality Assessment

Data integrity is assessed by examining the presence of missing values and duplicate observations. The `skim()` function is additionally employed to produce a comprehensive statistical profile of all variables. Confirming data completeness and consistency is essential before proceeding to exploratory and predictive analysis.

```
# Missing values
colSums(is.na(boston.df))
```

```
##    crim      zn   indus    chas     nox      rm     age     dis     rad     tax
##       0       0       0       0       0       0       0       0       0       0
## ptratio   black   lstat    medv
##       0       0       0       0
```

```r
# Duplicates
sum(duplicated(boston.df))
```

```
## [1] 0
```

```r
# Quick profile
skim(boston.df)
```

Table 2: Data summary

Name	boston.df
Number of rows	506
Number of columns	14
Column type frequency:	
numeric	14
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
crim	0	1	3.61	8.60	0.01	0.08	0.26	3.68	88.98	
zn	0	1	11.36	23.32	0.00	0.00	0.00	12.50	100.00	
indus	0	1	11.14	6.86	0.46	5.19	9.69	18.10	27.74	
chas	0	1	0.07	0.25	0.00	0.00	0.00	0.00	1.00	
nox	0	1	0.55	0.12	0.38	0.45	0.54	0.62	0.87	
rm	0	1	6.28	0.70	3.56	5.89	6.21	6.62	8.78	
age	0	1	68.57	28.15	2.90	45.02	77.50	94.07	100.00	
dis	0	1	3.80	2.11	1.13	2.10	3.21	5.19	12.13	
rad	0	1	9.55	8.71	1.00	4.00	5.00	24.00	24.00	
tax	0	1	408.24	168.54	187.00	279.00	330.00	666.00	711.00	
ptratio	0	1	18.46	2.16	12.60	17.40	19.05	20.20	22.00	
black	0	1	356.67	91.29	0.32	375.38	391.44	396.22	396.90	
lstat	0	1	12.65	7.14	1.73	6.95	11.36	16.96	37.97	
medv	0	1	22.53	9.20	5.00	17.02	21.20	25.00	50.00	

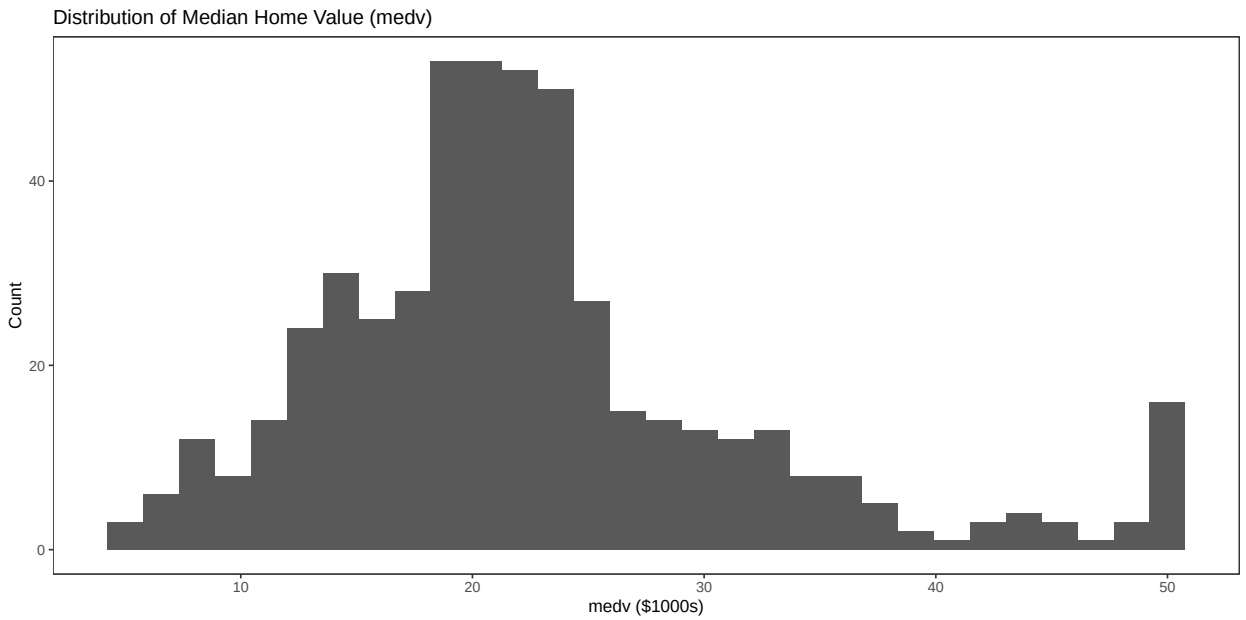
Comprehensive data validation confirmed that the dataset contains no missing values and no duplicate records, ensuring complete coverage across all predictors. The `skim()` profile further verified that all variables are numeric and span realistic ranges consistent with the dataset's documentation. While data completeness is excellent, the summary statistics suggest the presence of skewed predictors (e.g., crime rate) and potentially influential outliers. These characteristics are common in socioeconomic datasets and support the need for both robust linear and non-linear modeling approaches.

Exploratory Data Analysis (EDA)

Target variable distribution

The distribution of median housing values is visualized using a histogram. This visualization facilitates assessment of skewness, variability, and potential boundary effects, including the known upper cap of the `medv` variable in the dataset.

```r
ggplot(boston.df, aes(medv)) +
  geom_histogram(bins = 30) +
  labs(title = "Distribution of Median Home Value (medv)",
       x = "medv ($1000s)",
       y = "Count")
```



A histogram of `medv` shows a moderately skewed distribution with visible clustering near the upper limit of 50. This accumulation at the maximum value confirms the earlier observation of top-coding. The distribution suggests that while most properties cluster in the mid-range ($15k–$30k), a subset approaches the imposed ceiling. From a modeling perspective, this truncation can distort error behavior for high-value neighborhoods and should be considered when interpreting prediction performance.

Relationship with key predictors

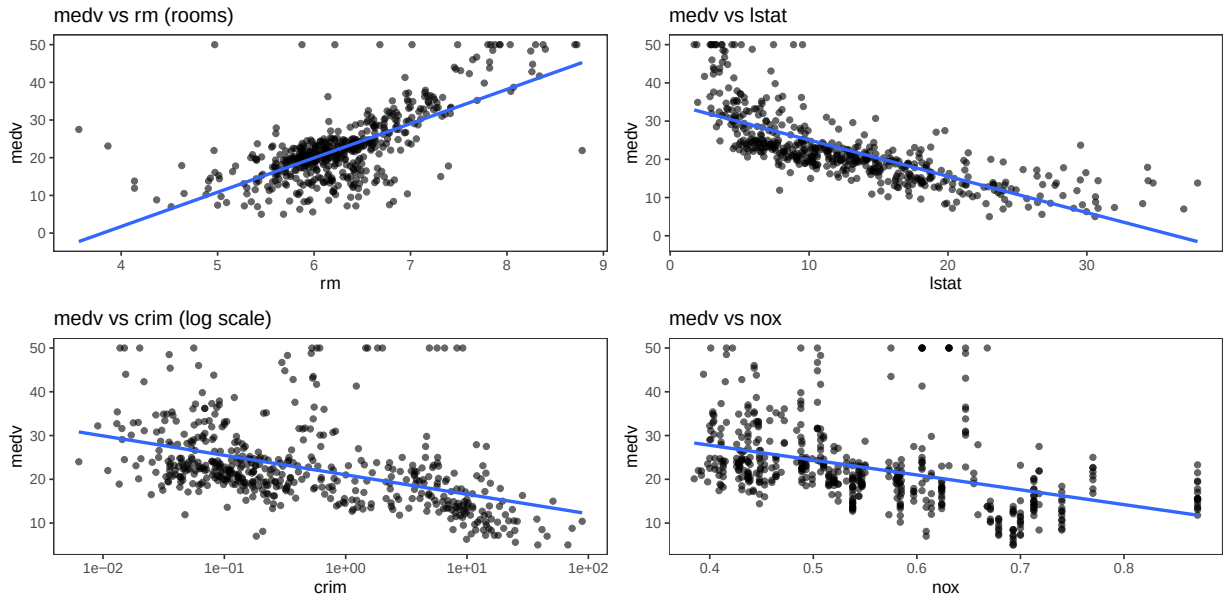
Scatterplots are generated to examine bivariate relationships between median housing value and selected predictors, including average number of rooms (`rm`), lower-status population percentage (`lstat`), crime rate (`crim`), and nitrogen oxide concentration (`nox`). Linear trend lines are included to illustrate directionality and strength of association. These visualizations provide initial evidence of potential predictive relationships.

```r
p1 <- ggplot(boston.df, aes(rm, medv)) + geom_point(alpha = 0.6) +
  geom_smooth(method="lm", se=FALSE) +
  labs(title="medv vs rm (rooms)")
p2 <- ggplot(boston.df, aes(lstat, medv)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method="lm", se=FALSE) +
  labs(title="medv vs lstat")
p3 <- ggplot(boston.df, aes(crim, medv)) +
  geom_point(alpha = 0.6) + scale_x_log10() +
  geom_smooth(method="lm", se=FALSE) +
  labs(title="medv vs crim (log scale)")
p4 <- ggplot(boston.df, aes(nox, medv)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method="lm", se=FALSE) +
```

```
  labs(title="medv vs nox")

(p1 | p2) / (p3 | p4)
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



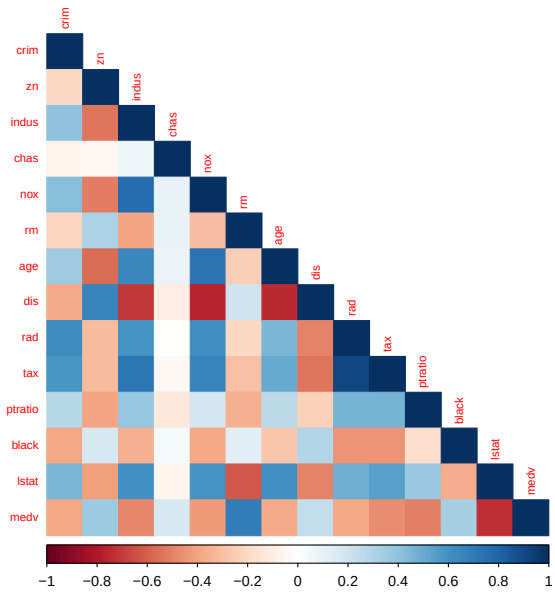
Scatterplots examining relationships between `medv` and selected predictors reveal strong and meaningful patterns. The number of rooms per dwelling (`rm`) shows a clear positive linear relationship with housing value, indicating that larger homes command higher prices. Conversely, the percentage of lower-status population (`lstat`) exhibits a strong negative association, suggesting socioeconomic composition significantly influences property valuation. Crime rate (`crim`), visualized on a log scale due to skewness, demonstrates a generally negative relationship with home values, while nitric oxide concentration (`nox`) also correlates negatively, reflecting the economic cost of environmental pollution. These patterns confirm domain expectations and suggest that both structural and socioeconomic variables are primary drivers of housing prices.

Correlation Overview

A correlation matrix is computed for all numeric variables and visualized using a heatmap. This analysis identifies strong linear associations among predictors and highlights potential multicollinearity concerns that may influence regression-based modeling approaches.

```
num_df <- boston.df %>% select(where(is.numeric))
corr <- cor(num_df)

corrplot(corr, method = "color", type = "lower", tl.cex = 0.7)
```



The correlation heatmap highlights clusters of interrelated predictors, particularly among urban infrastructure and environmental variables. Several predictors display moderate to strong correlations, indicating potential multicollinearity. While this does not invalidate linear regression, it may affect coefficient stability and interpretability. The presence of correlated features justifies the inclusion of regularized regression (Elastic Net) and tree-based models (Random Forest), which are more resilient to multicollinearity and complex interactions.

Data Preparation

Train/ Test Split

The dataset is partitioned into training (80%) and testing (20%) subsets using stratified sampling based on the target variable. This separation enables unbiased evaluation of model generalization performance on unseen data.

```
set.seed(123)
idx <- createDataPartition(boston.df$medv, p = 0.8, list = FALSE)
train <- boston.df[idx, ]
test  <- boston.df[-idx, ]

dim(train); dim(test)
```

```
## [1] 407  14
```

```
## [1] 99 14
```

The dataset was partitioned into an 80% training set (407 observations) and a 20% test set (99 observations) using stratified sampling on the target variable. This ensures that model performance is evaluated on unseen data, preserving the integrity of generalization metrics. The holdout sample size is sufficient to produce reliable test estimates given the overall dataset size.

Preprocessing Recipe

Predictor variables are standardized through centering and scaling transformations estimated on the training data. The same transformations are subsequently applied to the test set to ensure consistency and to prevent data leakage. Standardization is particularly important for regularized regression techniques.

```
preproc <- preProcess(
  train %>% select(-medv),
  method = c("center", "scale")
)

x_train <- predict(preproc, train %>% select(-medv))
x_test  <- predict(preproc, test %>% select(-medv))

y_train <- train$medv
y_test  <- test$medv
```

Predictors were centered and scaled using parameters learned exclusively from the training set to prevent data leakage. The same transformation was applied to the test set, ensuring consistency across modeling stages. Standardization is particularly critical for regularized regression models, where coefficient shrinkage depends on comparable feature scales.

Modeling

Evaluation Setup (Cross-Validation)

A repeated 10-fold cross-validation strategy is specified to obtain stable and reliable estimates of model performance. Repetition reduces variance in resampling estimates and enhances robustness in model comparison.

Custom functions are defined to compute Root Mean Squared Error (`RMSE`), Mean Absolute Error (`MAE`), and Coefficient of Determination (`R^2`). These metrics are applied consistently across all candidate models to ensure comparability.

```
ctrl <- trainControl(
  method = "repeatedcv",
  number = 10,
  repeats = 3
)

rmse <- function(actual, pred) sqrt(mean((actual - pred)^2))
mae  <- function(actual, pred) mean(abs(actual - pred))
r2   <- function(actual, pred) cor(actual, pred)^2
```

Baseline Model (Mean Predictor)

A baseline model is constructed by predicting the mean of the training target variable for all test observations. This benchmark establishes a minimum performance threshold that subsequent predictive models are expected to exceed.

```
baseline_pred <- rep(mean(y_train), length(y_test))

baseline_metrics <- tibble(
  Model = "Baseline (Train Mean)",
  RMSE = rmse(y_test, baseline_pred),
  MAE  = mae(y_test, baseline_pred),
  R2   = r2(y_test, baseline_pred)
)
```

```
## Warning in cor(actual, pred): the standard deviation is zero
```

```
baseline_metrics
```

```
## # A tibble: 1 x 4
##   Model                  RMSE   MAE    R2
##   <chr>                 <dbl> <dbl> <dbl>
## 1 Baseline (Train Mean)  9.32  6.60    NA
```

A naive baseline model predicting the training mean for all test observations produced an `RMSE` of approximately 9.32 and an `MAE` of 6.60. The `R^2` metric was undefined because constant predictions yield zero variance in the predicted values. This baseline establishes a reference point: any meaningful predictive model must substantially outperform this error level.

Multiple Linear Regression

A multiple linear regression model is trained using all available predictors. Cross-validation is applied during training to estimate performance stability. Predictions are generated on the test dataset to assess out-of-sample performance.

```
lm_fit <- train(
  medv ~ ., data = train,
  method = "lm",
  trControl = ctrl
)

lm_pred <- predict(lm_fit, newdata = test)

lm_metrics <- tibble(
  Model = "Linear Regression",
  RMSE = rmse(y_test, lm_pred),
  MAE  = mae(y_test, lm_pred),
  R2   = r2(y_test, lm_pred)
)

lm_fit
```

```
## Linear Regression
##
## 407 samples
##  13 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold, repeated 3 times)
## Summary of sample sizes: 367, 366, 366, 367, 366, 366, ...
## Resampling results:
##
##   RMSE      Rsquared   MAE
##   4.867445  0.7264844  3.42562
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

```
lm_metrics
```

```
## # A tibble: 1 x 4
##   Model              RMSE   MAE    R2
##   <chr>             <dbl> <dbl> <dbl>
## 1 Linear Regression  4.59  3.37 0.761
```

The ordinary least squares regression model demonstrated strong predictive capacity, achieving a test `RMSE` of approximately 4.59 and an `R^2` of 0.761. This indicates that the linear model explains roughly 76% of the variance in unseen housing values.

Diagnostics (Residuals)

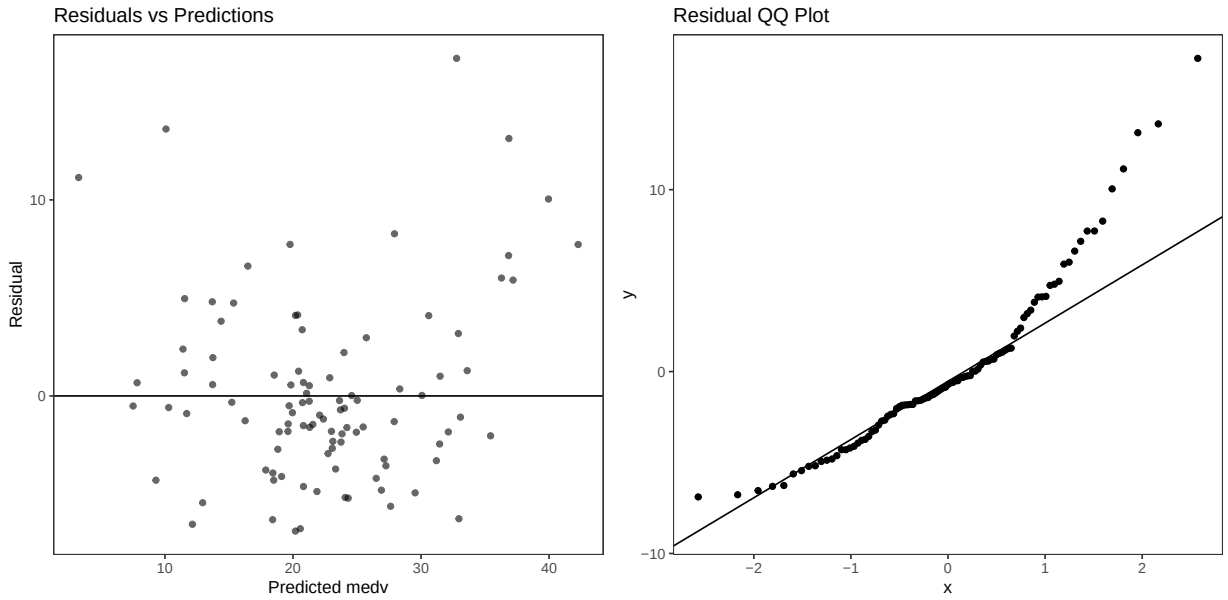
Diagnostic plots are produced to evaluate regression assumptions, including homoscedasticity and normality of residuals. Residual-versus-predicted plots assess systematic bias, while Q–Q plots evaluate deviation from normality.

```
res <- y_test - lm_pred

p_res1 <- ggplot(tibble(pred=lm_pred, res=res), aes(pred, res)) +
  geom_point(alpha=0.6) +
  geom_hline(yintercept = 0) +
  labs(title="Residuals vs Predictions",
       x="Predicted medv",
       y="Residual")

p_res2 <- ggplot(tibble(res=res), aes(sample=res)) +
  stat_qq() + stat_qq_line() +
  labs(title="Residual QQ Plot")

p_res1 | p_res2
```



Residual diagnostics show generally centered errors, though deviations in the tails suggest that linear assumptions may not fully capture complex patterns in the data. Nevertheless, the model offers solid interpretability and forms a robust benchmark for comparison.

Regularized Regression (Elastic Net via glmnet)

An Elastic Net regression model is trained to address potential multicollinearity and overfitting. A grid search over alpha (mixing parameter) and lambda (regularization strength) is conducted within the cross-validation framework to identify optimal hyperparameters.

```
grid_glmnet <- expand.grid(
  alpha  = seq(0, 1, by = 0.25),
  lambda = 10^seq(-3, 1, length.out = 40)
)

glmnet_fit <- train(
  x = as.matrix(x_train),
  y = y_train,
```

```
  method = "glmnet",
  trControl = ctrl,
  tuneGrid = grid_glmnet
)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
glmnet_pred <- predict(glmnet_fit, newdata = as.matrix(x_test))

glmnet_metrics <- tibble(
  Model = "Elastic Net (glmnet)",
  RMSE = rmse(y_test, glmnet_pred),
  MAE  = mae(y_test, glmnet_pred),
  R2   = r2(y_test, glmnet_pred)
)

glmnet_fit
```

```
## glmnet
##
## 407 samples
##  13 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold, repeated 3 times)
## Summary of sample sizes: 366, 367, 366, 367, 365, 367, ...
## Resampling results across tuning parameters:
##
##   alpha  lambda        RMSE      Rsquared   MAE
##   0.00    0.001000000  4.832917  0.7277185  3.355326
##   0.00    0.001266380  4.832917  0.7277185  3.355326
##   0.00    0.001603719  4.832917  0.7277185  3.355326
##   0.00    0.002030918  4.832917  0.7277185  3.355326
##   0.00    0.002571914  4.832917  0.7277185  3.355326
##   0.00    0.003257021  4.832917  0.7277185  3.355326
##   0.00    0.004124626  4.832917  0.7277185  3.355326
##   0.00    0.005223345  4.832917  0.7277185  3.355326
##   0.00    0.006614741  4.832917  0.7277185  3.355326
##   0.00    0.008376776  4.832917  0.7277185  3.355326
##   0.00    0.010608184  4.832917  0.7277185  3.355326
##   0.00    0.013433993  4.832917  0.7277185  3.355326
##   0.00    0.017012543  4.832917  0.7277185  3.355326
##   0.00    0.021544347  4.832917  0.7277185  3.355326
##   0.00    0.027283334  4.832917  0.7277185  3.355326
##   0.00    0.034551073  4.832917  0.7277185  3.355326
##   0.00    0.043754794  4.832917  0.7277185  3.355326
##   0.00    0.055410203  4.832917  0.7277185  3.355326
##   0.00    0.070170383  4.832917  0.7277185  3.355326
##   0.00    0.088862382  4.832917  0.7277185  3.355326
##   0.00    0.112533558  4.832917  0.7277185  3.355326
##   0.00    0.142510267  4.832917  0.7277185  3.355326
##   0.00    0.180472177  4.832917  0.7277185  3.355326
```

```
##   0.00    0.228546386  4.832917  0.7277185  3.355326
##   0.00    0.289426612  4.832917  0.7277185  3.355326
##   0.00    0.366524124  4.832917  0.7277185  3.355326
##   0.00    0.464158883  4.832917  0.7277185  3.355326
##   0.00    0.587801607  4.832917  0.7277185  3.355326
##   0.00    0.744380301  4.836849  0.7274231  3.355220
##   0.00    0.942668455  4.847351  0.7265883  3.356558
##   0.00    1.193776642  4.863034  0.7254231  3.362947
##   0.00    1.511775071  4.885406  0.7238610  3.372637
##   0.00    1.914481976  4.916719  0.7217748  3.387967
##   0.00    2.424462017  4.959320  0.7190457  3.412150
##   0.00    3.070290630  5.016077  0.7154868  3.447561
##   0.00    3.888155180  5.089893  0.7109021  3.497191
##   0.00    4.923882632  5.183249  0.7050648  3.563397
##   0.00    6.235507341  5.298286  0.6977548  3.648744
##   0.00    7.896522868  5.435376  0.6888018  3.754453
##   0.00   10.000000000  5.594498  0.6781015  3.881450
##   0.25    0.001000000  4.836438  0.7279316  3.421037
##   0.25    0.001266380  4.836438  0.7279316  3.421037
##   0.25    0.001603719  4.836438  0.7279316  3.421037
##   0.25    0.002030918  4.836438  0.7279316  3.421037
##   0.25    0.002571914  4.836438  0.7279316  3.421037
##   0.25    0.003257021  4.836438  0.7279316  3.421037
##   0.25    0.004124626  4.836438  0.7279316  3.421037
##   0.25    0.005223345  4.836438  0.7279316  3.421037
##   0.25    0.006614741  4.836438  0.7279316  3.421037
##   0.25    0.008376776  4.836438  0.7279316  3.421037
##   0.25    0.010608184  4.836438  0.7279316  3.421037
##   0.25    0.013433993  4.836438  0.7279316  3.421037
##   0.25    0.017012543  4.836126  0.7279642  3.420499
##   0.25    0.021544347  4.835448  0.7280174  3.418529
##   0.25    0.027283334  4.834691  0.7280719  3.416190
##   0.25    0.034551073  4.833752  0.7281342  3.413073
##   0.25    0.043754794  4.832779  0.7281915  3.409349
##   0.25    0.055410203  4.831802  0.7282406  3.405026
##   0.25    0.070170383  4.831070  0.7282552  3.400320
##   0.25    0.088862382  4.830387  0.7282608  3.394986
##   0.25    0.112533558  4.830098  0.7282192  3.389107
##   0.25    0.142510267  4.830765  0.7280725  3.382744
##   0.25    0.180472177  4.833037  0.7277600  3.376673
##   0.25    0.228546386  4.837666  0.7271960  3.371650
##   0.25    0.289426612  4.846338  0.7262152  3.368771
##   0.25    0.366524124  4.859747  0.7247570  3.370375
##   0.25    0.464158883  4.882315  0.7223572  3.382206
##   0.25    0.587801607  4.914495  0.7189612  3.405712
##   0.25    0.744380301  4.947600  0.7156659  3.431511
##   0.25    0.942668455  4.983606  0.7125137  3.459282
##   0.25    1.193776642  5.032040  0.7085892  3.493198
##   0.25    1.511775071  5.102265  0.7029653  3.545636
##   0.25    1.914481976  5.193345  0.6959931  3.615083
##   0.25    2.424462017  5.270615  0.6934212  3.673924
##   0.25    3.070290630  5.370804  0.6918821  3.753471
##   0.25    3.888155180  5.512105  0.6903937  3.869596
##   0.25    4.923882632  5.707120  0.6887406  4.020335
```

```
##   0.25    6.235507341  5.978255  0.6860340  4.221638
##   0.25    7.896522868  6.332114  0.6827414  4.480222
##   0.25   10.000000000  6.773510  0.6804070  4.802855
##   0.50    0.001000000  4.836713  0.7279107  3.421480
##   0.50    0.001266380  4.836713  0.7279107  3.421480
##   0.50    0.001603719  4.836713  0.7279107  3.421480
##   0.50    0.002030918  4.836713  0.7279107  3.421480
##   0.50    0.002571914  4.836713  0.7279107  3.421480
##   0.50    0.003257021  4.836713  0.7279107  3.421480
##   0.50    0.004124626  4.836713  0.7279107  3.421480
##   0.50    0.005223345  4.836713  0.7279107  3.421480
##   0.50    0.006614741  4.836713  0.7279107  3.421480
##   0.50    0.008376776  4.836713  0.7279107  3.421480
##   0.50    0.010608184  4.836697  0.7279150  3.421381
##   0.50    0.013433993  4.836031  0.7279773  3.419769
##   0.50    0.017012543  4.835262  0.7280360  3.417458
##   0.50    0.021544347  4.834449  0.7280919  3.414616
##   0.50    0.027283334  4.833635  0.7281389  3.411206
##   0.50    0.034551073  4.832891  0.7281695  3.407232
##   0.50    0.043754794  4.832380  0.7281668  3.402730
##   0.50    0.055410203  4.831926  0.7281538  3.397857
##   0.50    0.070170383  4.831984  0.7280757  3.392448
##   0.50    0.088862382  4.833136  0.7278771  3.386425
##   0.50    0.112533558  4.836118  0.7274809  3.380509
##   0.50    0.142510267  4.842461  0.7267166  3.377234
##   0.50    0.180472177  4.852353  0.7255827  3.377136
##   0.50    0.228546386  4.870951  0.7234989  3.382360
##   0.50    0.289426612  4.899792  0.7202916  3.400647
##   0.50    0.366524124  4.929304  0.7169687  3.423896
##   0.50    0.464158883  4.958964  0.7138715  3.449014
##   0.50    0.587801607  4.994496  0.7104918  3.476491
##   0.50    0.744380301  5.050397  0.7051602  3.513228
##   0.50    0.942668455  5.121662  0.6985085  3.569930
##   0.50    1.193776642  5.175958  0.6951532  3.612571
##   0.50    1.511775071  5.237230  0.6935732  3.662030
##   0.50    1.914481976  5.329118  0.6915465  3.744926
##   0.50    2.424462017  5.464350  0.6886591  3.850862
##   0.50    3.070290630  5.660907  0.6846162  3.992614
##   0.50    3.888155180  5.921404  0.6820803  4.185898
##   0.50    4.923882632  6.291648  0.6764095  4.465401
##   0.50    6.235507341  6.797329  0.6622056  4.842805
##   0.50    7.896522868  7.400063  0.6465935  5.293495
##   0.50   10.000000000  8.103378  0.6364385  5.834225
##   0.75    0.001000000  4.836690  0.7279229  3.421542
##   0.75    0.001266380  4.836690  0.7279229  3.421542
##   0.75    0.001603719  4.836690  0.7279229  3.421542
##   0.75    0.002030918  4.836690  0.7279229  3.421542
##   0.75    0.002571914  4.836690  0.7279229  3.421542
##   0.75    0.003257021  4.836690  0.7279229  3.421542
##   0.75    0.004124626  4.836690  0.7279229  3.421542
##   0.75    0.005223345  4.836690  0.7279229  3.421542
##   0.75    0.006614741  4.836690  0.7279229  3.421542
##   0.75    0.008376776  4.836710  0.7279226  3.421431
##   0.75    0.010608184  4.836068  0.7279748  3.419615
```

```
##   0.75    0.013433993  4.835350  0.7280287  3.417250
##   0.75    0.017012543  4.834629  0.7280729  3.414369
##   0.75    0.021544347  4.833932  0.7281066  3.410953
##   0.75    0.027283334  4.833425  0.7281114  3.407017
##   0.75    0.034551073  4.833008  0.7280994  3.402481
##   0.75    0.043754794  4.832856  0.7280528  3.397572
##   0.75    0.055410203  4.833426  0.7279187  3.392220
##   0.75    0.070170383  4.835443  0.7276272  3.386307
##   0.75    0.088862382  4.840003  0.7270548  3.381470
##   0.75    0.112533558  4.847571  0.7261622  3.380053
##   0.75    0.142510267  4.862229  0.7244927  3.383295
##   0.75    0.180472177  4.888362  0.7215788  3.395470
##   0.75    0.228546386  4.917226  0.7182479  3.417167
##   0.75    0.289426612  4.946246  0.7149746  3.440273
##   0.75    0.366524124  4.974950  0.7119673  3.465805
##   0.75    0.464158883  5.020260  0.7074320  3.496109
##   0.75    0.587801607  5.083766  0.7010340  3.543089
##   0.75    0.744380301  5.139083  0.6960326  3.588260
##   0.75    0.942668455  5.188635  0.6934659  3.629347
##   0.75    1.193776642  5.259491  0.6907996  3.693384
##   0.75    1.511775071  5.361902  0.6872161  3.781261
##   0.75    1.914481976  5.506020  0.6833882  3.888623
##   0.75    2.424462017  5.706964  0.6799755  4.035583
##   0.75    3.070290630  6.008052  0.6729193  4.260038
##   0.75    3.888155180  6.446079  0.6567268  4.599301
##   0.75    4.923882632  6.981752  0.6436352  4.985612
##   0.75    6.235507341  7.676483  0.6316435  5.497847
##   0.75    7.896522868  8.559988  0.5763886  6.199007
##   0.75   10.000000000  9.093140        NaN  6.656521
##   1.00    0.001000000  4.836615  0.7279357  3.421427
##   1.00    0.001266380  4.836615  0.7279357  3.421427
##   1.00    0.001603719  4.836615  0.7279357  3.421427
##   1.00    0.002030918  4.836615  0.7279357  3.421427
##   1.00    0.002571914  4.836615  0.7279357  3.421427
##   1.00    0.003257021  4.836615  0.7279357  3.421427
##   1.00    0.004124626  4.836615  0.7279357  3.421427
##   1.00    0.005223345  4.836615  0.7279357  3.421427
##   1.00    0.006614741  4.836603  0.7279371  3.421349
##   1.00    0.008376776  4.836203  0.7279680  3.419911
##   1.00    0.010608184  4.835522  0.7280182  3.417621
##   1.00    0.013433993  4.834848  0.7280593  3.414836
##   1.00    0.017012543  4.834214  0.7280879  3.411511
##   1.00    0.021544347  4.833748  0.7280907  3.407690
##   1.00    0.027283334  4.833427  0.7280702  3.403222
##   1.00    0.034551073  4.833368  0.7280155  3.398398
##   1.00    0.043754794  4.834073  0.7278690  3.393130
##   1.00    0.055410203  4.836418  0.7275414  3.387423
##   1.00    0.070170383  4.841459  0.7269127  3.382811
##   1.00    0.088862382  4.849269  0.7259970  3.382069
##   1.00    0.112533558  4.865842  0.7241078  3.386840
##   1.00    0.142510267  4.893700  0.7209779  3.400972
##   1.00    0.180472177  4.921924  0.7176848  3.422384
##   1.00    0.228546386  4.950280  0.7144407  3.445367
##   1.00    0.289426612  4.979325  0.7113398  3.471195
```

```
##   1.00    0.366524124  5.028411  0.7062696  3.503732
##   1.00    0.464158883  5.087574  0.7002001  3.548916
##   1.00    0.587801607  5.142487  0.6949330  3.594093
##   1.00    0.744380301  5.192871  0.6919565  3.637952
##   1.00    0.942668455  5.264424  0.6885141  3.704611
##   1.00    1.193776642  5.364412  0.6843656  3.788128
##   1.00    1.511775071  5.496356  0.6811964  3.889111
##   1.00    1.914481976  5.698927  0.6759074  4.040179
##   1.00    2.424462017  6.007488  0.6648269  4.271241
##   1.00    3.070290630  6.441553  0.6454123  4.608152
##   1.00    3.888155180  6.970209  0.6357967  4.974705
##   1.00    4.923882632  7.733097  0.6044262  5.530644
##   1.00    6.235507341  8.679876  0.5699586  6.299511
##   1.00    7.896522868  9.093140        NaN  6.656521
##   1.00   10.000000000  9.093140        NaN  6.656521
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 0.25 and lambda = 0.1125336.
```

```
glmnet_metrics
```

```
## # A tibble: 1 x 4
##   Model                 RMSE   MAE    R2
##   <chr>                <dbl> <dbl> <dbl>
## 1 Elastic Net (glmnet)  4.59  3.37 0.764
```

Elastic Net regression, tuned via cross-validation, selected an optimal combination of alpha = 0.25 and lambda 0.1125. On the test set, it achieved an `RMSE` nearly identical to linear regression (4.59) with a slightly improved `R^2` of 0.764.

Coefficient (Interpretable Drivers)

Coefficients from the optimal Elastic Net model are extracted and ranked according to magnitude. This analysis highlights the most influential predictors contributing to variations in median housing value, enhancing interpretability.

```
best <- glmnet_fit$bestTune
best
```

```
##    alpha    lambda
## 61  0.25 0.1125336
```

```
final_glmnet <- glmnet_fit$finalModel
coef_mat <- coef(final_glmnet, s = best$lambda)

coef_df <- tibble(
  term = rownames(as.matrix(coef_mat)),
  estimate = as.numeric(as.matrix(coef_mat))
) %>%
  filter(term != "(Intercept)") %>%
  arrange(desc(abs(estimate)))

head(coef_df, 12)
```

```
## # A tibble: 12 x 2
##    term    estimate
##    <chr>      <dbl>
##  1 lstat     -3.89
##  2 rm         2.64
##  3 dis       -2.64
##  4 rad        2.17
##  5 ptratio   -2.00
##  6 nox       -1.70
##  7 tax       -1.46
##  8 black      0.912
##  9 zn         0.724
## 10 crim      -0.669
## 11 chas       0.612
## 12 indus     -0.198
```

Coefficient analysis identified `lstat` and `rm` as the most influential predictors, consistent with EDA findings. While performance gains over OLS were marginal, Elastic Net provides improved coefficient stability in the presence of correlated predictors, reinforcing confidence in variable importance interpretations.

Random Forest (Non-linear + Interactions)

A Random Forest model is trained to capture non-linear relationships and higher-order interactions among predictors. Hyperparameter tuning is conducted via cross-validation to determine the optimal number of variables sampled at each split (`mtry`).

```
set.seed(42)
rf_fit <- train(
  medv ~ ., data = train,
  method = "rf",
  trControl = ctrl,
  tuneLength = 8
)

rf_pred <- predict(rf_fit, newdata = test)

rf_metrics <- tibble(
  Model = "Random Forest",
  RMSE = rmse(y_test, rf_pred),
  MAE  = mae(y_test, rf_pred),
  R2   = r2(y_test, rf_pred)
)

rf_fit
```

```
## Random Forest
##
## 407 samples
##  13 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold, repeated 3 times)
```

```
## Summary of sample sizes: 366, 367, 367, 366, 365, 367, ...
## Resampling results across tuning parameters:
##
##   mtry  RMSE      Rsquared   MAE
##    2    3.730540  0.8500965  2.504164
##    3    3.455975  0.8689108  2.336131
##    5    3.286445  0.8780783  2.237428
##    6    3.257238  0.8800641  2.218811
##    8    3.257554  0.8779761  2.221670
##    9    3.277066  0.8762192  2.233907
##   11    3.332730  0.8710574  2.267585
##   13    3.370363  0.8674099  2.290992
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 6.
```

```
rf_metrics
```

```
## # A tibble: 1 x 4
##   Model          RMSE   MAE    R2
##   <chr>         <dbl> <dbl> <dbl>
## 1 Random Forest  3.10  2.12 0.896
```

The Random Forest model substantially outperformed all linear approaches. With an optimal mtry of 6, the model achieved a test `RMSE` of approximately 3.10 and an `R^2` of 0.896, explaining nearly 90% of the variance in housing prices. This improvement indicates the presence of meaningful non-linear relationships and interactions that linear models cannot fully capture.

Feature Importane (Random Forest)

Variable importance scores are extracted from the final Random Forest model and visualized to identify the most influential predictors. This provides insight into non-linear driver contributions within the ensemble framework.

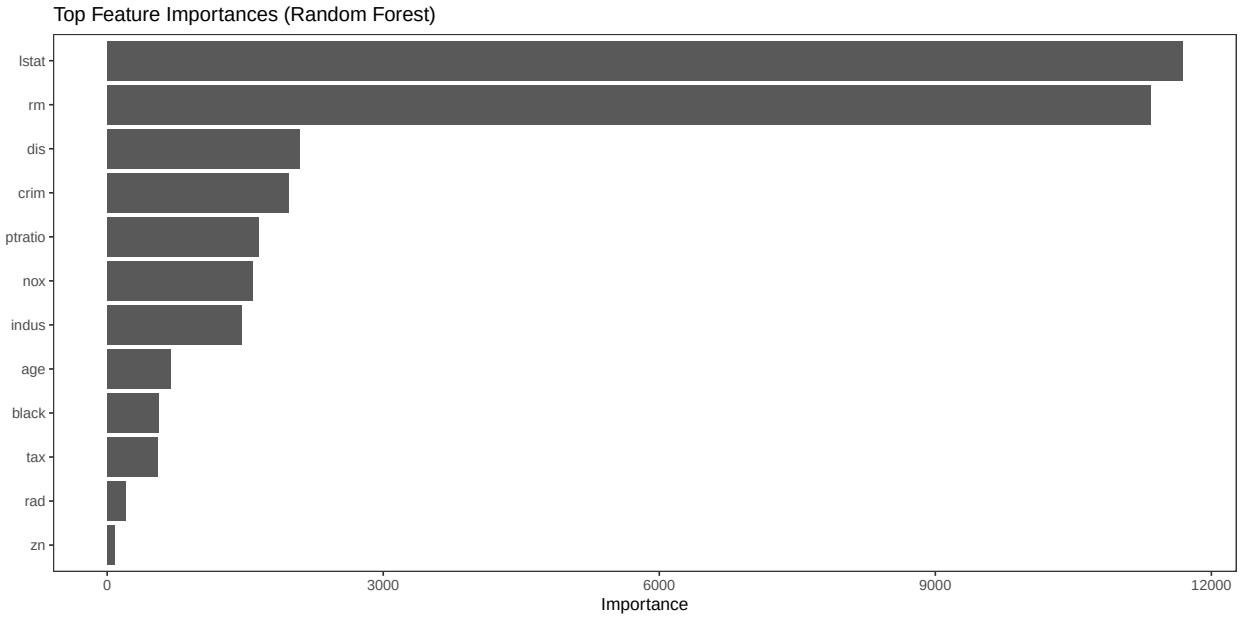
```
rf_final <- rf_fit$finalModel
imp <- randomForest::importance(rf_final)
imp_df <- tibble(
  Feature = rownames(imp),
  Importance = imp[,1]
) %>% arrange(desc(Importance))

head(imp_df, 12)
```

```
## # A tibble: 12 x 2
##    Feature Importance
##    <chr>        <dbl>
##  1 lstat      11687.
##  2 rm         11335.
##  3 dis         2095.
##  4 crim        1977.
##  5 ptratio     1644.
##  6 nox         1578.
```

```
##  7 indus       1458.
##  8 age          690.
##  9 black        555.
## 10 tax          544.
## 11 rad          198.
## 12 zn            84.1
```

```
ggplot(imp_df %>% slice_max(Importance, n=12),
       aes(reorder(Feature, Importance), Importance)) +
  geom_col() +
  coord_flip() +
  labs(title="Top Feature Importances (Random Forest)",
       x=NULL,
       y="Importance")
```



Feature importance rankings again identified `lstat`, `rm`, `dis`, and `crim` as dominant predictors, confirming consistency across modeling paradigms.

Evaluation

Compare Models on Test Set

Performance metrics from all candidate models are consolidated into a single comparison table and ranked by RMSE. This systematic evaluation identifies the model that demonstrates superior predictive accuracy on the test dataset.

```
results <- bind_rows(baseline_metrics, lm_metrics,
                     glmnet_metrics, rf_metrics) %>%
  arrange(RMSE)

results
```

```
## # A tibble: 4 x 4
##   Model                  RMSE   MAE     R2
##   <chr>                 <dbl> <dbl>  <dbl>
## 1 Random Forest          3.10  2.12  0.896
## 2 Linear Regression      4.59  3.37  0.761
## 3 Elastic Net (glmnet)   4.59  3.37  0.764
## 4 Baseline (Train Mean)  9.32  6.60 NA
```

When ranked by `RMSE`, Random Forest clearly outperforms Linear Regression and Elastic Net, while the baseline model performs significantly worse. The comparative analysis demonstrates that flexible ensemble methods yield superior predictive accuracy for this problem, particularly given the mixture of socioeconomic and environmental variables.

Prediction Quality Plot (Best Model)

A scatterplot of actual versus predicted values is generated for the best-performing model. The 45-degree reference line facilitates visual assessment of prediction accuracy and systematic deviation.

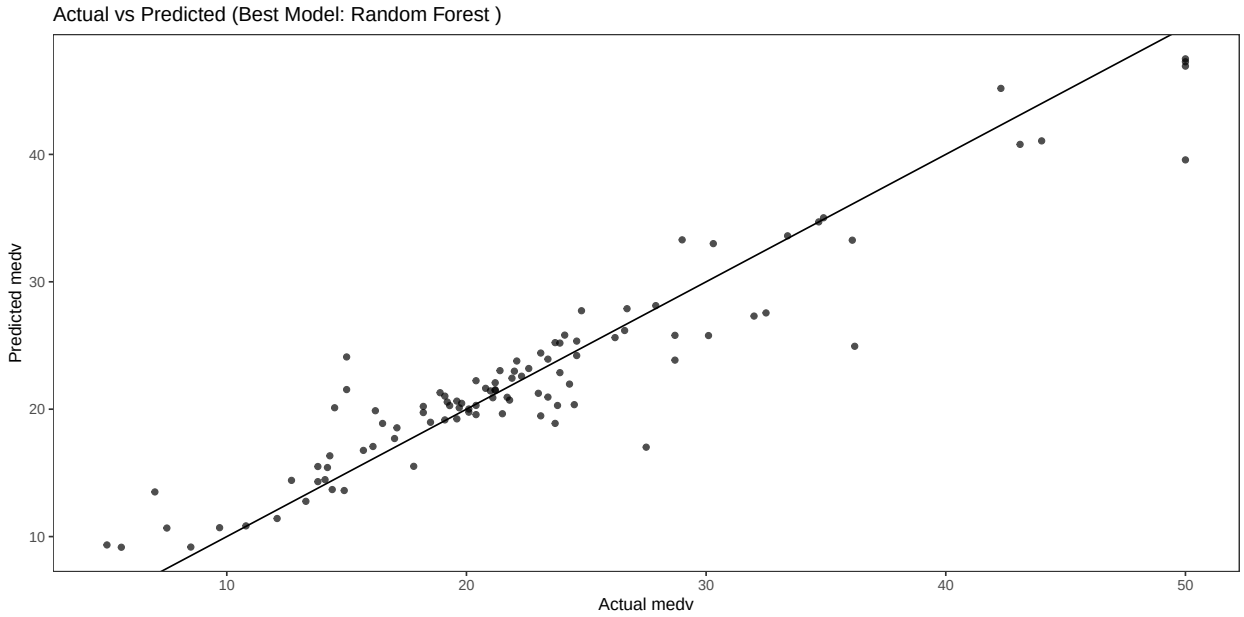
```
best_model <- results$Model[1]
best_model
```

```
## [1] "Random Forest"
```

```
best_pred <- switch(
  best_model,
  "Baseline (Train Mean)" = baseline_pred,
  "Linear Regression" = lm_pred,
  "Elastic Net (glmnet)" = glmnet_pred,
  "Random Forest" = rf_pred
)

plot_df <- tibble(
  actual = y_test,
  predicted = best_pred
)
```

```
ggplot(plot_df, aes(actual, predicted)) +
  geom_point(alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0) +
  labs(
    title = paste("Actual vs Predicted (Best Model:", best_model, ")"),
    x = "Actual medv",
    y = "Predicted medv"
  )
```



The predicted-versus-actual plot for the best-performing model shows strong alignment along the 45-degree reference line, indicating good calibration. However, some underprediction is observed for high-value tracts, likely influenced by the dataset's top-coded ceiling.

Error Analysis (Where the model misses)

Prediction errors are computed and ranked by absolute magnitude to identify observations where the model exhibits the largest deviations. A histogram of residuals is produced to assess distributional symmetry and potential bias.

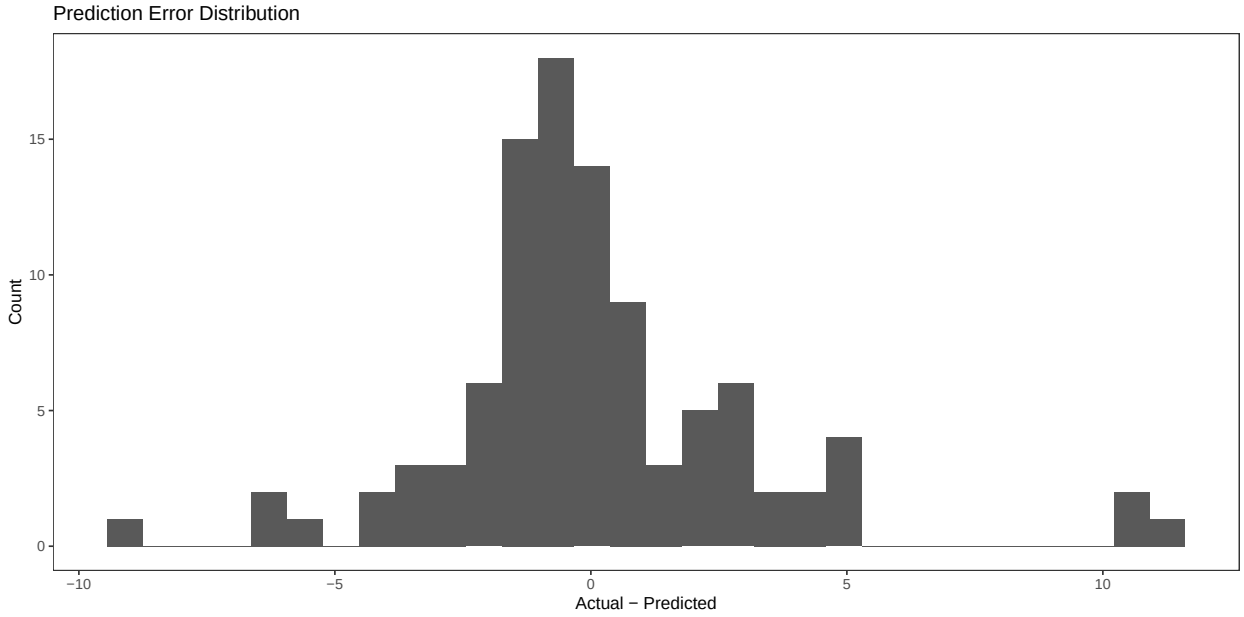
```
plot_df <- plot_df %>%
  mutate(error = actual - predicted,
         abs_error = abs(error)) %>%
  arrange(desc(abs_error))

head(plot_df, 10)
```

```
## # A tibble: 10 x 4
##    actual predicted error abs_error
##     <dbl>     <dbl> <dbl>     <dbl>
##  1   36.2      24.9 11.3      11.3
##  2   27.5      17.0 10.5      10.5
##  3   50        39.6 10.4      10.4
```

```
##  4   15        24.1 -9.10      9.10
##  5   15        21.5 -6.54      6.54
##  6    7        13.5 -6.50      6.50
##  7   14.5      20.1 -5.61      5.61
##  8   32.5      27.6  4.94      4.94
##  9   28.7      23.9  4.85      4.85
## 10   23.7      18.9  4.82      4.82
```

```
ggplot(plot_df, aes(error)) +
  geom_histogram(bins = 30) +
  labs(title="Prediction Error Distribution",
       x="Actual - Predicted",
       y="Count")
```



Error analysis reveals that the largest deviations are concentrated among extreme observations, with absolute errors occasionally exceeding 10 (in $1,000s). The residual distribution remains approximately centered, suggesting no major systemic bias.

Deployment

A reusable scoring function is implemented to enable prediction on new, unseen data. The function verifies predictor consistency with the training dataset and applies the appropriate model-specific transformations prior to generating predictions.

The scoring function is demonstrated on a subset of test observations to simulate real-world deployment. This confirms that the trained model can be operationalized for practical use.

```
score_medv <- function(newdata, best_model_name = best_model) {
  # Ensure columns match training structure
  if (!all(names(train %>% select(-medv)) %in% names(newdata))) {
    stop("newdata must include all predictor columns used in training.")
  }
  newdata <- newdata %>% select(names(train %>% select(-medv)))

  pred <- switch(
    best_model_name,
    "Linear Regression" = predict(lm_fit, newdata = newdata),
    "Elastic Net (glmnet)" = {
      x_new <- predict(preproc, newdata)
      predict(glmnet_fit, newdata = as.matrix(x_new))
    },
    "Random Forest" = predict(rf_fit, newdata = newdata),
    rep(mean(y_train), nrow(newdata))
  )
  return(as.numeric(pred))
}

# Example: score the first 3 rows of the test set predictors
example_new <- test %>% select(-medv) %>% slice(1:3)
score_medv(example_new)
```

```
## [1] 34.70179 25.79722 18.88134
```

A reusable score_medv() function was implemented to facilitate production deployment. The function validates predictor availability, applies necessary preprocessing (for regularized models), and returns numeric predictions. Demonstration outputs for sample records confirm successful integration. This modular design enables straightforward extension into dashboards, APIs, or batch-scoring workflows, completing the CRISP-DM lifecycle from exploration to deployment readiness.

Conclusion

References